

Splines

Reading: You should read Chapter 5 in *The Elements of Statistical Learning*.

Splines are another popular approach to nonlinear regression. The basic idea is that we replace the input variable x with additional variables that are transformations of x and then use linear models in the new space.

Suppose that $f(\vec{x})$ is the non-linear function of the explanatory variables that we wish to model. We take

$$f(\vec{x}) = \sum_{m=1}^M \beta_m h_m(\vec{x})$$

Where the β_m 's are coefficients that we must calculate and the $h_m(\vec{x})$'s are transformations of $\vec{x}$. This is called a linear basis expansion in $\vec{x}$ because it is linear in the functions $h_m(\vec{x})$. This is point of splines: we replace a non-linear function with a sum of linear ones.

If we take

$$h_m(\vec{x}) = x_m, \quad m = 1..p$$

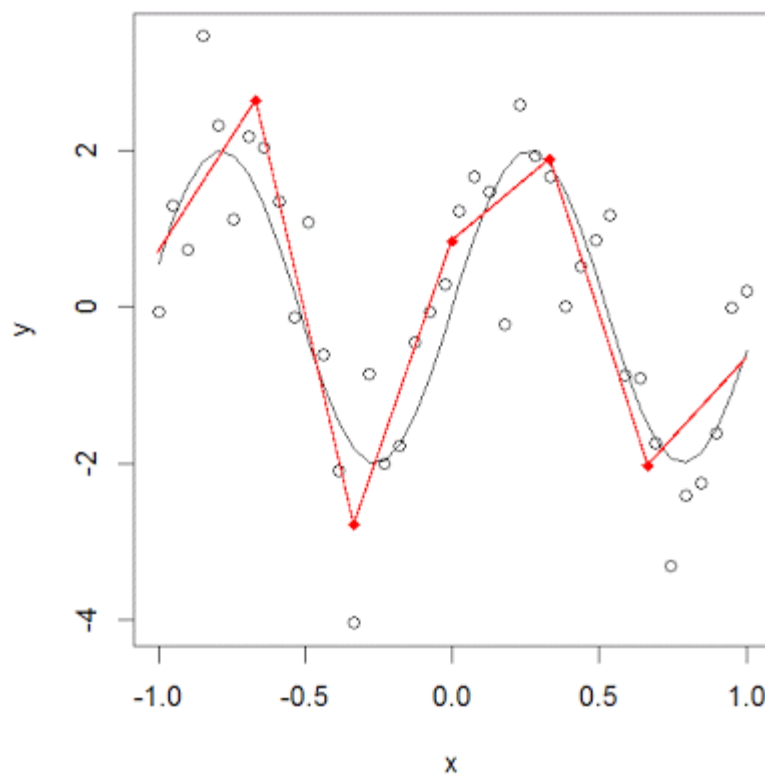
Then this gives the regular linear model

$$f(\vec{x}) = \sum_{m=1}^M \beta_m x_m$$

Piecewise polynomials and splines

For simplicity, assume that $\bar{x}$ is one-dimensional. Thus, we can take $\bar{x} = x$. A piecewise polynomial is obtained by dividing the domain of x into contiguous intervals and representing $f(x)$ in each one by a polynomial.

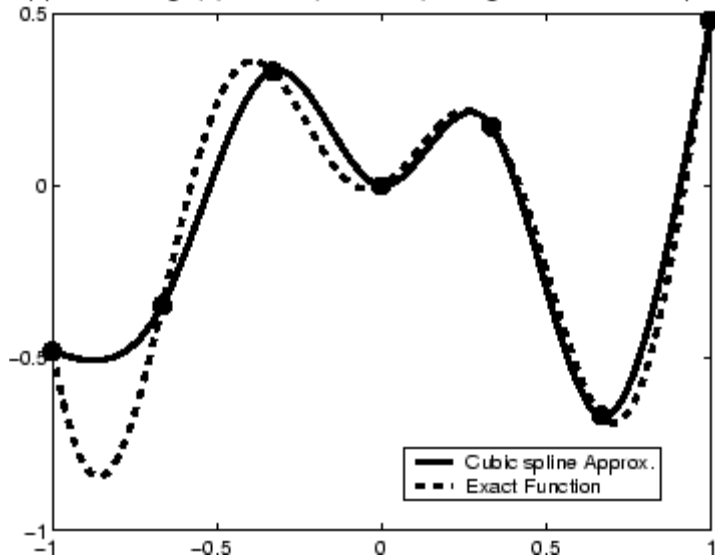
For example, the figure below shows a case where the curve is split into six regions and a straight line is fit in each one:



(image from <http://www.r-bloggers.com/estimating-continuous-piecewise-linear-regression/>)

In the example below, the function is again split into six regions, but cubic polynomials are fit in each region:

Approximating $f(x) = x \sin(2\pi x + 1)$ using Natural cubic splines



(image from <http://www.physics.arizona.edu/~restrepo/475A/Notes/sourcea-/node35.html>)

Go To Board for basic theory

Fitting Splines in R

There are many possible parameterizations of the splines. The power series splines shown previously are not great choices computationally because large powers often lead to large round-off errors.

A very popular choice are B-splines:

Take knots $e_1, e_2, \dots, e_k$

The knots are the points that mark the boundaries between the different regions in which we fit the curves.

Define boundary knots e_0 and e_{k+1} which typically bracket the domain over which x is defined. Take M as the order of splines that we are using.

Next define the following τ 's:

$$\tau_1 \leq \tau_2 \leq \dots \tau_M \leq e_0$$

$$\tau_{j+M} = e_j \quad j = 1..k$$

$$e_{k+1} \leq \tau_{k+M+1} \leq \tau_{k+M+2} \leq \dots \leq \tau_{k+2M}$$

The actual values of $\tau_1 \dots \tau_M$ and $\tau_{k+M+1} \dots \tau_{k+2M}$ are arbitrary. In practice, they are typically taken as equal to e_0 and e_{k+1} , respectively.

Define $B(i,m)(x)$ as the i th B-spline basis function of order m at point x . It is defined recursively:

$$\begin{aligned} B(i,1)(x) &= 1 \quad \text{if } \tau_i \leq x \leq \tau_{i+1} \\ &= 0 \quad \text{otherwise} \end{aligned}$$

for $i=1..M, k+M+1$ to $K+2M$

$$B(i,m)(x) = \frac{(x - \tau_i)B(i, m-1)(x)}{(\tau_{i+m-1} - \tau_i)} + \frac{(\tau_{i+m} - x)B(i+1, m-1)(x)}{(\tau_{i+m} - \tau_{i+1})}$$

The spline functions are in the package `splines`:

library(splines)

the function `bs` implements B-splines:

?bs

Take 3 knots and cubic splines. We can 1) either specify the knot positions with argument **knot** or 2) specify the **df** and it chooses the knot locations.

By default, it does not fit an intercept (can change this with the argument **intercept=T**). Thus, we get back one **df**

With three knots, the **df** should be

$df = (4 \text{ regions}) * (4 \text{ parameters for cubic splines}) - (3 \text{ knots}) * (3 \text{ constraints per knot}) - (1 \text{ for no intercept}) = 6$

```
basis1<-bs(dat1[,1],df=6)
```

```
basis1
```

```
> basis1<-bs(dat1[,1],df=6)
```

```
> basis1
```

```
      1      2      3      4      5      6
[1,] 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000
[2,] 0.383101852 0.0374228395 0.0007716049 0.0000000000 0.000000000 0.000000000
[3,] 0.564814815 0.1327160494 0.0061728395 0.0000000000 0.000000000 0.000000000
[4,] 0.593750000 0.2604166667 0.0208333333 0.0000000000 0.000000000 0.000000000
[5,] 0.518518519 0.3950617284 0.0493827160 0.0000000000 0.000000000 0.000000000
[6,] 0.387731481 0.5111882716 0.0964506173 0.0000000000 0.000000000 0.000000000
[7,] 0.250000000 0.5833333333 0.1666666667 0.0000000000 0.000000000 0.000000000
[8,] 0.144675926 0.5929783951 0.2615740741 0.0007716049 0.000000000 0.000000000
[9,] 0.074074074 0.5493827160 0.3703703704 0.0061728395 0.000000000 0.000000000
[10,] 0.031250000 0.4687500000 0.4791666667 0.0208333333 0.000000000 0.000000000
[11,] 0.009259259 0.3672839506 0.5740740741 0.0493827160 0.000000000 0.000000000
[12,] 0.001157407 0.2611882716 0.6412037037 0.0964506173 0.000000000 0.000000000
[13,] 0.000000000 0.1666666667 0.6666666667 0.1666666667 0.000000000 0.000000000
[14,] 0.000000000 0.0964506173 0.6412037037 0.2611882716 0.001157407 0.000000000
[15,] 0.000000000 0.0493827160 0.5740740741 0.3672839506 0.009259259 0.000000000
[16,] 0.000000000 0.0208333333 0.4791666667 0.4687500000 0.031250000 0.000000000
[17,] 0.000000000 0.0061728395 0.3703703704 0.5493827160 0.074074074 0.000000000
[18,] 0.000000000 0.0007716049 0.2615740741 0.5929783951 0.144675926 0.000000000
[19,] 0.000000000 0.000000000 0.1666666667 0.5833333333 0.250000000 0.000000000
[20,] 0.000000000 0.000000000 0.0964506173 0.5111882716 0.387731481 0.00462963
[21,] 0.000000000 0.000000000 0.0493827160 0.3950617284 0.518518519 0.03703704
[22,] 0.000000000 0.000000000 0.0208333333 0.2604166667 0.593750000 0.12500000
[23,] 0.000000000 0.000000000 0.0061728395 0.1327160494 0.564814815 0.29629630
[24,] 0.000000000 0.000000000 0.0007716049 0.0374228395 0.383101852 0.57870370
[25,] 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 1.000000000
```

```
attr("degree")
```

```
[1] 3
```

```
attr("knots")
```

```
25% 50% 75%
```

```
3 6 9
```

```
attr("Boundary.knots")
```

```
[1] 0 12
attr("intercept")
[1] FALSE
attr("class")
[1] "bs" "basis" "matrix"
```

This returns a basis matrix for representing the family of piecewise polynomials with the specified interior knots. The "knots" attribute tells which knots were chosen. If you specify the df, then *bs()* chooses the knots. It places the knots at the quantiles.

The matrix that is returned is the values of the basis functions at each x point. For this example we have six basis functions, so the matrix is N by 6.

For example, *basis1[2,1]* is the value (0.383) of the first basis function at *x=0.5*. Notice how each x point has four non-zero values and also note that the non-zero values switch at the knots. The first set of points have three non-zero values because the intercept is zero.

We can use the basis matrix to specify a linear model. We are specifying *PreyEaten* as a linear model with 6 variables:

```
spmod1<-lm(PreyEaten~bs(PreyDens,df=6),data=dat1)
summary(spmod1)
plot(dat1[,1],fitted(spmod1),type="l")
points(dat1)
```

```
> summary(spmod1)
```

Call:

```
lm(formula = PreyEaten ~ bs(PreyDens, df = 6), data = dat1)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.84819	-0.17901	0.03716	0.24165	0.53329

Coefficients:

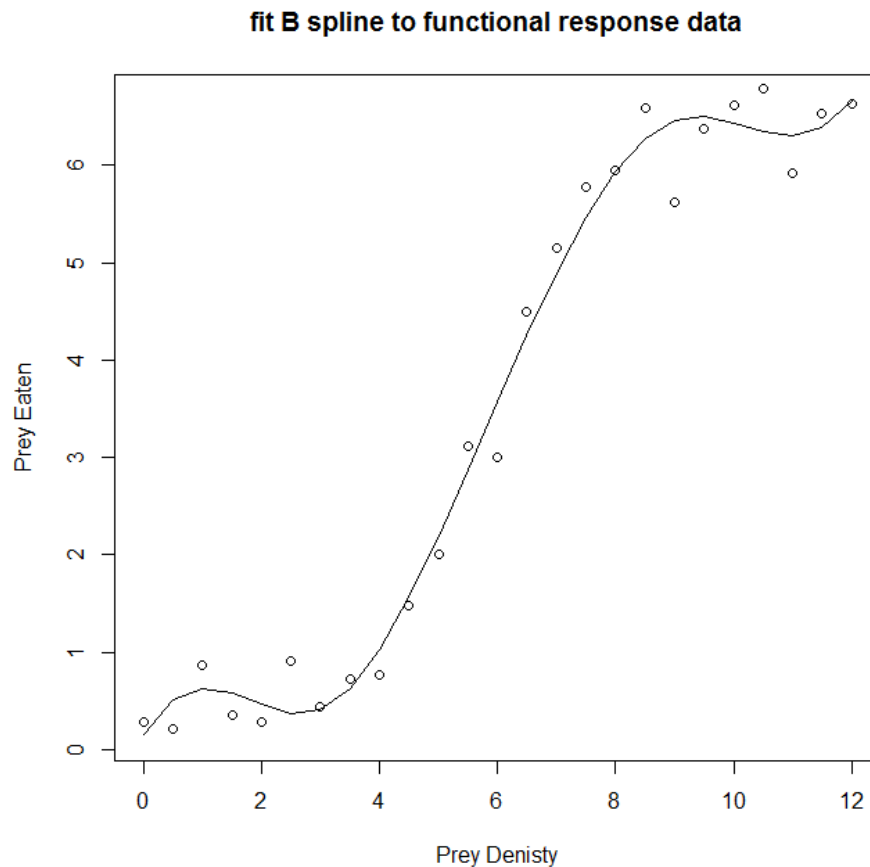
	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.1526	0.3400	0.449	0.6589
bs(PreyDens, df = 6)1	1.0450	0.6879	1.519	0.1461
bs(PreyDens, df = 6)2	-1.0252	0.5336	-1.921	0.0707 .
bs(PreyDens, df = 6)3	3.5538	0.5748	6.183	7.76e-06 ***
bs(PreyDens, df = 6)4	7.3620	0.5746	12.812	1.75e-10 ***
bs(PreyDens, df = 6)5	5.7061	0.5982	9.539	1.84e-08 ***
bs(PreyDens, df = 6)6	6.5149	0.4777	13.637	6.27e-11 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3727 on 18 degrees of freedom

Multiple R-squared: 0.9852, Adjusted R-squared: 0.9803

F-statistic: 199.6 on 6 and 18 DF, p-value: 1.831e-15



It doesn't look like it is doing great at the ends (certainly the upturn of the curve at the upper end would not be what we would expect biologically).

Try four knots:

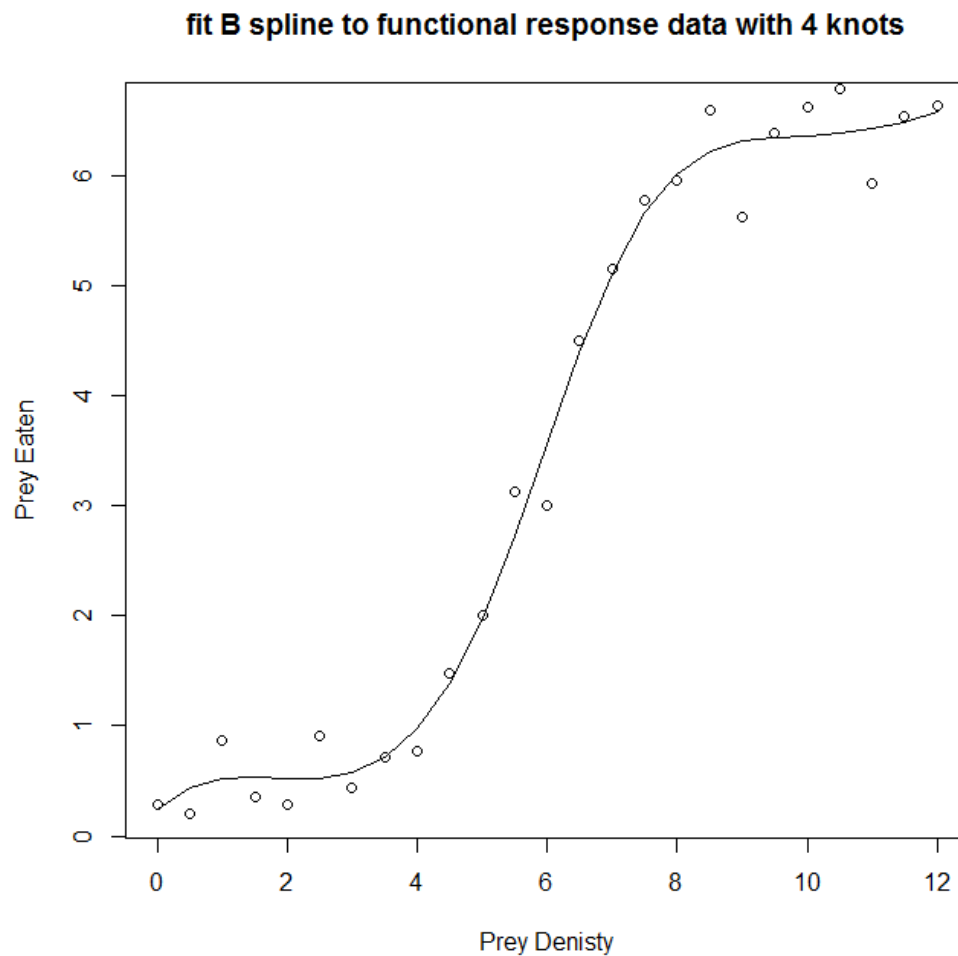
$df = 5 \times 4 - 4 \times 3 - 1 = 7$

```
spmod2<-lm(PreyEaten~bs(PreyDens,df=7),data=dat1)
```

```
summary(spmod2)
```

```
plot(dat1[,1],fitted(spmod2),type="l")
```

```
points(dat1)
```



Try five knots:

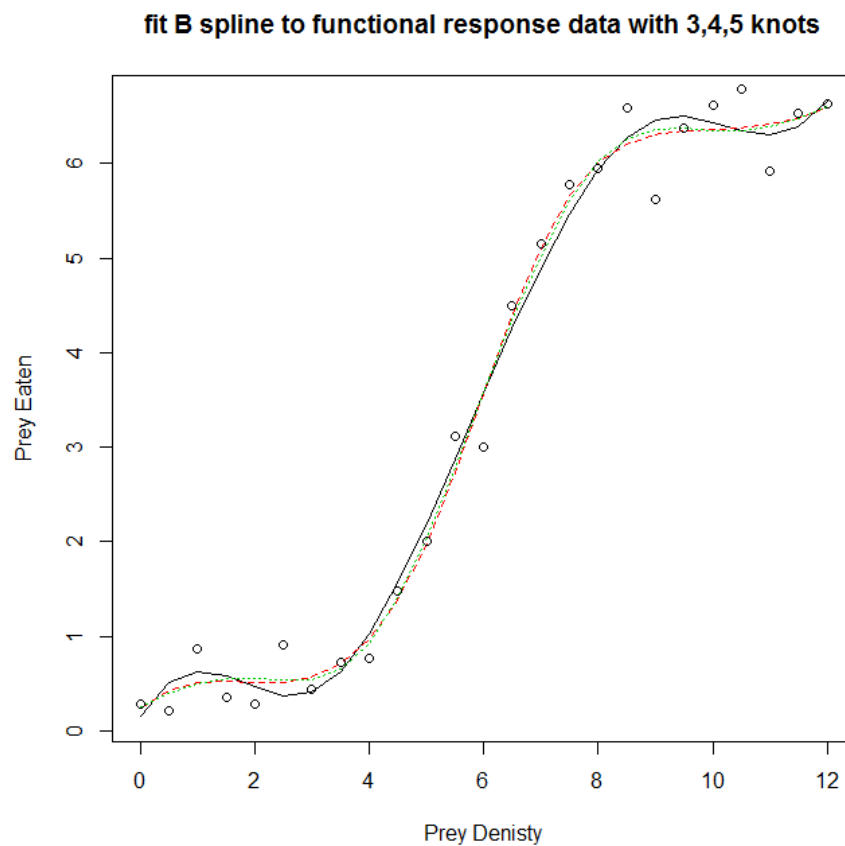
$df = 5 \times 4 - 1 = 8$

```
spmod3<-lm(PreyEaten~bs(PreyDens,df=8),data=dat1)
```

```
summary(spm3)
plot(dat1[,1],fitted(spm3),type="l")
points(dat1)

par(mfrow=c(1,1))
matplot(dat1[,1],cbind(cbind(fitted(spm1),fitted(spm2)),fitted(spm3)),type="l")
points(dat1)

#black is 3 knots
#green is 4 knots
#red is 5 knots
```



Both 4 and 5 knots seem to do quite well, although the behavior at the ends still seems questionable

Note that the **bs(PreyDens,df=7)** is simply interpreted as a data matrix similarly to any call to **lm**. **lm** doesn't "know" that this is output from *bs()*. The original one dimensional x data is converted to a set of basis functions. *lm()* treats these each as separate predictor variables.

Exercise

Compare the fits of the functions **nls**, **loess**, and **bs** to the exercise data. Put all three fits on the same plot. Do for cases of standard deviation of residuals in the data of 0.5 and 3.